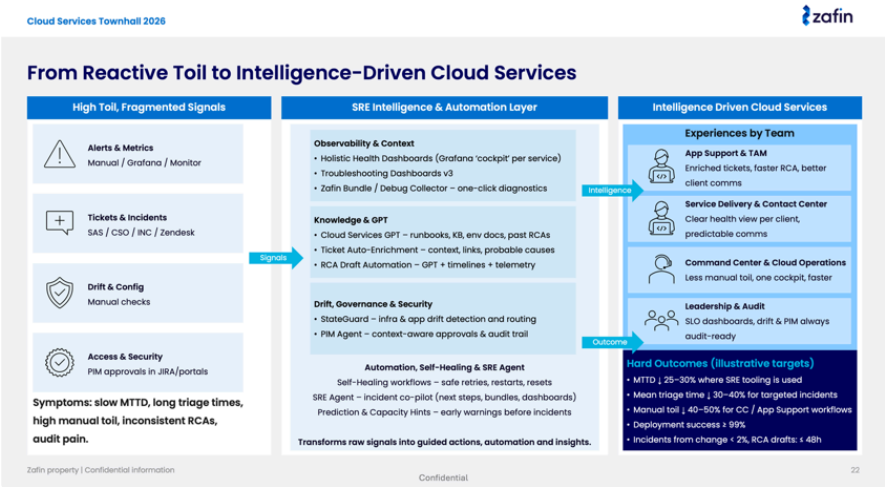
Cloud Services Automation Roadmap: 2026



1) Reactive Support Agent Experience

Persona

Reactive Support Agent (L1/L2 + SRE-assist)

Goal: **Shrink time-to-triage** and **time-to-mitigation** by auto-collecting evidence, classifying the fault domain (app vs infra), and guiding next actions.

Experience 1: "Customer incident → automated triage → SRE recommendation"

Trigger

1. **Customer submits ticket in Zendesk**
2. Zendesk auto-creates **Jira Incident** (with customer, env, time window, severity, impacted capability)

Step-by-step flow (what the user experiences)

Step 1 — Instant "Incident Console" link appears (in Jira) - Ticket Auto-enrichment

- A single URL opens the **Troubleshooting Cockpit (Grafana)** scoped to:
  - customer/project
  - cluster/namespace
  - service(s)
  - time window around "incident reported at T0"

Step 2 — Two parallel "evidence engines" fire automatically

1. **Infra Debug Bundle Agent (system-side)**
   - Collects cluster & platform evidence into a packaged "debug bundle"
   - Typical contents:
     - Kubernetes events, node status, pod restarts/OOMKills
     - resource pressure signals, autoscaler activity
     - network/DNS anomalies
     - ingress/egress, LB health, certificate expiry

        - Azure infra signals relevant to your architecture (AKS, VMSS, storage, LB, Key Vault access errors, etc.)
        - Log Analytics queries run automatically for the period (T0-15m to T0+30m)
    - Stores bundle in a controlled location (blob/storage) and registers metadata back into Jira.
2. **Application Context Agent (app-side)**
    - Pulls "what's deployed for this customer right now":
        - release version, feature flags/config deltas, recent changes
        - dependency health (DB, queue, external integrations)
        - trace exemplars (Tempo) and error patterns
    - Searches "similar incidents":
        - prior Jira incidents, postmortems/RCA docs in Confluence
        - known errors/alerts mapping (runbooks)
    - Outputs:
        - suspected app failure mode(s)
        - likely impacted components
        - proposed immediate checks (top 5)

Step 3 — Orchestrator Agent decides "where to look first"

- Consumes both streams and produces a **fault-domain classification**:
    - **Infra-likely** (e.g., node pressure, kube DNS outage, AKS event, cert expiry, storage throttling)
    - **App-likely** (e.g., regression, config drift, DB lock, long running queries, memory leak)
    - **Mixed / cascading** (app causes infra symptoms or vice versa)
- Posts back into Jira:
    - classification + confidence score
    - "first 3 actions" checklist
    - suggested assignee route (L2 vs SRE vs L2 + Plat/Prod Eng)

Step 4 — SRE Agent enters when debug bundle is ready

- SRE Agent analyzes debug bundle + current metrics/traces/logs and produces:
    - recommended mitigation (restart strategy, scale action, rollback candidate, traffic shift, circuit breaker, config change)
    - risk assessment and rollback steps
    - "what to monitor next 30 minutes" list
- Can generate a ready-to-run set of commands/scripts *as suggestions* (**human executes**).

Step 5 — RCA Agent starts in parallel

- From the moment severity is set:
    - opens a "Living RCA Draft" in Confluence
    - continuously appends:
        - timeline (ticket time, detection time, first response, mitigations)
        - evidence links (Grafana panels, Log Analytics queries, Tempo traces)
        - customer impact statement template
    - generates **customer-facing updates** every N minutes (for L1 to paste/send)

Experience 2: "Engineer investigating → instant troubleshooting cockpit"

Trigger

- Any engineer opens a Jira incident (internal or customer) and clicks **"Open Cockpit"**

What the cockpit provides (opinionated and time-scoped)

1-click investigative workflow

- "At a glance" panels:
  - golden signals per service (latency, traffic, errors, saturation)
  - customer-scoped error rate
  - last deploy marker, config change marker
- "Pinpoint the moment" tools:
  - auto-zoom to incident time window
  - correlated event overlays (deploys, scaling, failovers, cert renewal, DB maintenance, batch jobs)
- Deep links:
  - Logs: pre-filtered Log Analytics/Grafana Loki queries
  - Traces: Tempo trace exemplars for failed requests
  - Metrics: Mimir/Prometheus series filtered to namespace/service/customer tags

SRE Agent button

- "Analyze current view":
  - summarizes what's abnormal
  - suggests next query/panel to open
  - suggests which runbook applies
  - flags if evidence quality is insufficient ("enable debug logging for X for next 20 minutes")

Outputs / Reactive KPI targets

- MTTA (acknowledge): minutes → near-instant
- MTTT (triage): hours → **<15 minutes**
- MTTR (restore): **30–50% reduction** for common failure modes
- Consistency: fewer "tribal knowledge" escalations
- Auditability: every action linked to evidence and timeline

2) Proactive Support Agent Experience

Persona

Proactive Support Agent (SRE-driven + platform health)

Goal: prevent incidents and reduce customer impact by continuously scanning clusters/tenants, predicting failures, and executing playbooks (human-approved).

Experience 1: "Always-on health cockpit + predictive risk + recommended actions"

Trigger

- Every X minutes (e.g., 5–15 min), Proactive Agent runs a **Health Sweep**:
  - per cluster
  - per customer rg's / vm's - MRC/DB (where safe/possible)
  - per critical dependency (DB, queues, storage)

What the cockpit shows (executive-simple but engineer-actionable)

Top screen: Fleet Health

- "Green / Yellow / Red" by cluster and by customer
- "Risks forming" queue (not yet an incident)
- Trend anomalies:

- memory creep
- CPU saturation leading indicators
- error budget burn rate
- latency regression since last release

Deep screen: Predictive Findings

- Each finding includes:
  - signal + supporting charts
  - confidence + time-to-impact estimate
  - recommended action + runbook link
  - “blast radius” (which customers/services)

Example scenarios (to be expanded)

Database risk detection (to work with dba’s)

- “High bloat / lock contention rising”
- “Vacuum / autovacuum not keeping up”
- “Wraparound risk indicators rising”

Outputs:

- recommended DB maintenance steps
- query candidates / top offending sessions
- suggested configuration adjustments
- pre-written customer advisory if impact is likely

Container memory creep

- Detects monotonic growth + GC pressure + restart history

Outputs:

- recommend rollout restart window / memory limit tuning
- point to recent code paths via traces
- create proactive Jira task before customer impact

Cluster / node resource pressure (Cloud Ops to expand)

- Predicts when cluster hits saturation given trend slope

Outputs:

- recommend node pool scale, pod right-sizing, or noisy neighbour isolation
- suggests cost-aware option (scale temporarily vs tune limits)
- disk pressure, log over growing
- PDB

How proactive becomes operational (not just “dashboards”)

Proactive Agent produces “Action Tickets,” not just alerts

- Creates Jira tasks/incidents based on severity thresholds
- Attaches evidence links and recommended steps
- Routes to:
  - Cloud Ops (infra)
  - App team (code/config)

- Support L2 (customer comms prep)
- Alert SRE

RCA Agent supports prevention reporting

- Weekly "Top 10 prevented incidents" summary
- "Most common predictors" and "runbook coverage gaps"
- Customer-facing narrative for major preventative maintenance (optional)

3) IAM/PIM Agent Experience (Security & Compliance)

Persona

IAM/PIM Agent (Security Ops + Cloud Ops assist)

Goal: **Automate PIM approvals**, improve compliance and reduce manual review load by validating every Azure PIM request against **ticket legitimacy**, **scope appropriateness**, **policy constraints**, **risk signals**, and **separation-of-duties** with full auditability.

Experience 1: "PIM request → automated validation → approve/route/deny"

Trigger

- A user raises a **Privileged Identity Management (PIM)** request in **Azure (Entra/Azure AD PIM)** with:
  - requested role
  - scope (subscription/RG/resource)
  - duration
  - justification + **ticket reference**

Step-by-step flow (what the approver experiences)

Step 1 — Agent immediately enriches the request ("PIM Review Packet")

The agent compiles a standardized evidence pack:

- **Ticket validation**
  - Ticket exists (Jira reference)
  - Ticket is **open / active** (not Done/Closed/Cancelled)
  - Ticket is **recent** (not stale; within defined window)
  - Ticket requester matches PIM requester or is explicitly authorized (e.g., on-call)
- **Access appropriateness**
  - Role matches job function + allowed catalog ("Role Allowlist" by team)
  - Scope matches what the ticket needs (no over-scoped requests)
  - Duration is within policy (default durations by role + emergency constraints)
- **Risk checks**
  - Break-glass vs standard elevation
  - MFA present, device compliance, sign-in risk (if available)
  - Unusual behavior signals (e.g., request at odd hours, rare role, new user, abnormal frequency)
- **Separation-of-duties (SoD)**
  - Conflicting roles detection (e.g., requester can approve own changes + deploy + alter audit)
  - "Two-person rule" requirements for high-privilege roles
- **Prior context**
  - Similar requests historically (pattern/abuse detection)
  - Prior denials/violations

Step 2 — Policy Engine scores the request

Agent produces:

- **Decision recommendation:** Approve / Route for human approval / Deny
- **Confidence + rationale ("why")**
- **Required controls** (e.g., forced MFA, shorter duration, restricted scope, require peer approval)

Step 3 — Orchestrator executes the workflow

- **Low-risk, policy-compliant**: auto-approve *only if the request follows* the specific roles/scopes/time limits.
- **Medium/high-risk**: routes to the right approver group with the full "PIM Review Packet"
- **Non-compliant**: denies with a clear reason + auto-creates a Jira task for remediation / correct ticketing

Step 4 — Audit & traceability

Every decision produces an immutable record:

- request → evidence → rationale → approver (human or policy) → final action

  This aligns with the same roadmap guardrail posture (auditability, RBAC, deterministic evidence).

Experience 2: "Approver cockpit" (for leadership + day-to-day ops)

A **Grafana IAM/PIM Cockpit** (security dashboard pack) similar to your troubleshooting cockpit concept, but focused on access governance. (Same "cockpit dashboard" pattern you already use for incidents.)

At-a-glance panels

- PIM request volume by hour/day
- % auto-approved vs human-approved vs denied
- Top requested roles/scopes + top requesters/teams
- SLA: time-to-decision (policy vs human)
- Risk distribution (low/med/high)
- Exceptions / policy drift (e.g., frequent scope escalations)

Deep views

- "Most suspicious requests" queue (risk score + reason)
- SoD conflict list
- Ticket mismatch list (invalid ticket, stale ticket, closed ticket)
- Emergency/break-glass activations report

4) Agent Roles and Responsibilities

Core agents (minimum viable set)

1. **Support Intake Agent (Zendesk/Jira)**
   - normalizes ticket data (customer, env, timestamps, severity hints)
2. **Infra Debug Bundle Agent**
   - gathers platform evidence + runs standard queries
3. **Application Context Agent**
   - gathers release/config context + prior incident similarity
4. **Orchestrator / Classifier Agent**
   - decides likely domain + routes workflow

5. **SRE Analysis Agent**
   - interprets bundle + live telemetry, proposes mitigations
6. **RCA & Comms Agent**
   - customer updates + timeline + post-incident draft RCA
7. **IAM/PIM Agent**
   - Auto-approve PIM request against ticket legitimacy, scope, policy, and risk.

5) Tooling Integration Map

System of record

- **Zendesk**: customer entry point
- **Jira**: incident lifecycle + action tracking
- **Confluence**: RCA + runbooks + known issues

Observability + evidence

- **Azure Log Analytics / Loki**: logs + query automation for time window
- **Grafana**: cockpit dashboards + investigative views
- **Mimir / Prometheus**: metrics and alert inputs
- **Tempo**: tracing evidence (exemplars, slow/error traces)

6) Deliverables

- The agents and orchestrations
- The debug bundle pipeline (collection → storage → metadata → access control)
- Grafana dashboard pack (Troubleshooting + Proactive cockpit)
- Ticket/RCA automation and comms templates

Deliverable 1 — Reactive "Incident-to-Recommendation" workflow

- Zendesk/Jira integration + incident console links
- Automated debug bundle (time-scoped, customer-scoped)
- App context extraction + similarity matching (Jira/Confluence history)
- Orchestrator routing + confidence
- SRE recommendation output format standardized

Deliverable 2 — Proactive "Fleet Health & Prediction" workflow

- Health sweep engine + anomaly detection rules
- Risk register in Grafana + Jira auto-tasking
- Runbook linking and "actionability scoring"

Deliverable 3 — RCA + Customer Communication automation

- Living RCA draft in Confluence
- periodic customer update drafts (L1-ready)
- final RCA draft template with evidence links and timeline

Deliverable 4 — IAM/PIM Governance Automation

- Azure PIM event ingestion + decision workflow
- Jira ticket validation and linkage
- Policy engine (allowlists, scope rules, duration rules, SoD rules)
- Risk scoring + anomaly detection

- Grafana IAM/PIM cockpit dashboard pack
- Audit trail storage + export for compliance

7) Guardrails - non-negotiable

Security / customer isolation

- Strict tenant scoping in queries and dashboards
- Role-based access (L1 vs L2 vs SRE vs Eng)
- Redaction of secrets/PII in bundles and drafts
- Immutable audit log: who viewed what, who triggered what, every decision includes evidence and “why”
- **RBAC**: only approved approver groups can override/force-approve high-risk requests

Reliability of the system itself

- If agents fail, workflows degrade gracefully (no “all-or-nothing”)
- Evidence capture must be deterministic and repeatable
-

> **Human-in-the-loop**
>
> - Agents recommend; humans execute
> - Clear confidence and “why” for every classification

8) MVP Phasing

Phase 1 (Reactive foundation)

- Incident Console link + Troubleshooting Cockpit
- Infra Debug Bundle automation
- RCA/Comms drafting
- PIM Automation

Phase 2 (Domain classification + app similarity)

- Orchestrator classification
- App Context Agent with prior incident matching
- PIM risk scoring

Phase 3 (Proactive health sweep)

- Fleet cockpit + risk queue
- Jira action ticket automation for top predictors
- PIM dashboards

Phase 4 (Predictive + optimization loop)

- trend-based forecasting
- runbook coverage analytics + continuous improvement